Discrete Structures Project

Project Title: Implementation of Cryptography, and Caesar's Cipher
Group Members: Ameen Dahani (IU05 0124 0198)
Kashif Raza (IU05 0124 0303)
Bahzad Shah (IU05 0124 0233)
Mudassir Hssain (IU05 0124 0193)
Course Name: Discrete Structures
Institution Name: Iqra University
Instructor's Name: Miss Yamna

Table of Contents

Introduction 2
- Background of the problem: 2
- Objective of the project: 2
- Scope of the project: 2
- Why you chose this project: 2

System Requirements 2
Project Design and Methodology 3
- Flowchart 3
- Algorithm Description: 4

Implementation 5
- 🔧 Key Programming Constructs Used: 5
- Code Structure: 6

Code Listing 7
Output Screenshot 10
- Explanation: 10

Conclusion 11
- Summary: 11
- Limitations: 12
- Future Enhancements: 12

References 12

Introduction

Background of the problem:

Cryptography is a crucial technique for securing data in the digital era. One of the earliest and simplest methods of encryption is Caesar's Cipher, a type of substitution cipher.

Objective of the project:

To implement a simple cryptography technique using Caesar's Cipher in C++.

Scope of the project:

This project demonstrates the encoding and decoding of messages using Caesar's Cipher, highlighting how characters are shifted within the alphabet.

Why you chose this project:

It offers a hands-on way to understand the basics of encryption and strengthens C++ programming skills.

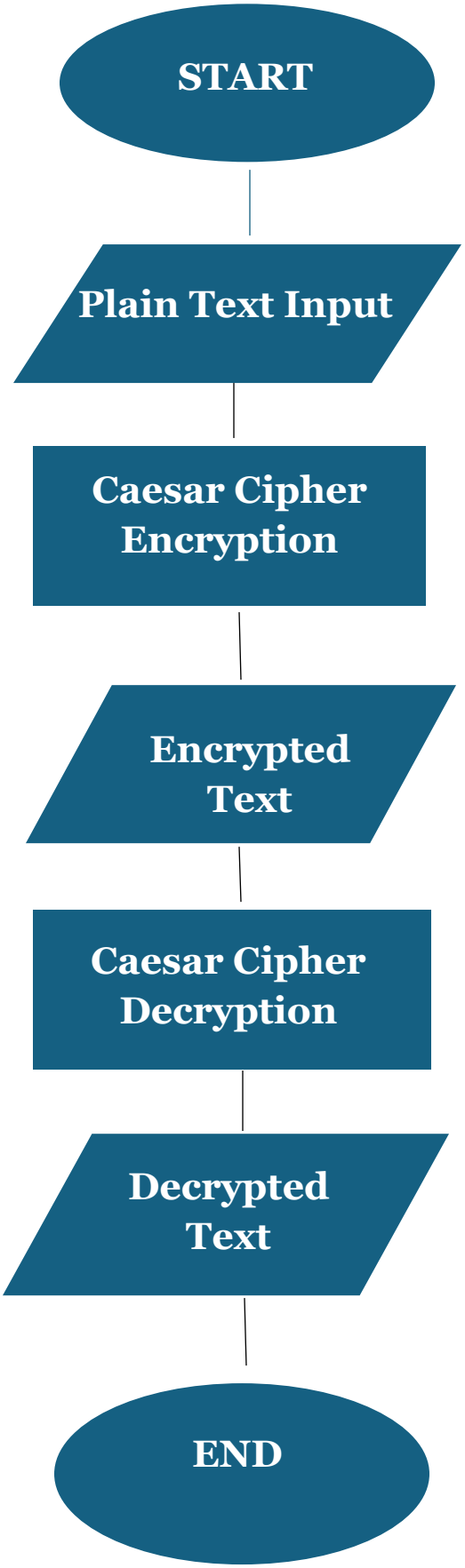
System Requirements

Software Requirements:

- Operating System: Windows 10 or Linux Ubuntu
- IDE: Code::Blocks / Visual Studio Code
- Compiler: GCC / MinGW

Project Design and Methodology

Flowchart:



Algorithm Description:

Step 1: Start

Step 2: Input the Plaintext and Shift Key

- Prompt the user to enter a message (Plaintext).
- Prompt the user to enter a shift value between 1 and 25.

Step 3: Validate Shift Key

- If the shift is less than 1 or greater than 25, display an error and terminate the program.

Step 4: Encrypt the Plaintext

- Initialize an empty string EncryptedText.
- For each character c in Plaintext:
    - If c is an **alphabetic character**:
        - Determine the **base** ('A' for uppercase, 'a' for lowercase).
        - Apply the Caesar Cipher shift using the formula:
          ShiftedChar = (c - base + shift) % 26 + base
        - Append ShiftedChar to EncryptedText.
    - Else:
        - Append the non-alphabetic character as-is to EncryptedText.

Step 5: Display the Encrypted Text

Step 6: Decrypt the Encrypted Text

- Use the same algorithm as encryption but with shift = (26 - shift).
- Store the result in DecryptedText.

Step 7: Display the Decrypted Text

Step 8: End

Implementation

🔧 Key Programming Constructs Used:

1. **Header Files:**
   - #include <iostream> – for input and output.
   - #include <string> – to use the string class.
2. **Namespaces:**
   - using namespace std; – avoids writing std:: repeatedly.
3. **Functions:**
   - encrypt(string text, int shift) – handles the encryption logic.
   - decrypt(string text, int shift) – reverses the encryption using a reverse shift.
   - main() – the entry point of the program, handles user input/output.
4. **Control Structures:**
   - if...else – to check if a character is alphabetic and to validate the shift value.
   - for loop – to iterate through each character in the input string.
5. **String Handling:**
   - Use of getline(cin, message) to read full lines (including spaces).
   - Use of character operations and ASCII values for encryption logic.
6. **Character Functions:**
   - isalpha(c) – checks if a character is a letter.
   - isupper(c) – checks if a character is uppercase.
7. **Modular Arithmetic:**
   - Used to wrap the character shift within the 26-letter alphabet.

Code Structure:

🔗 Explanation of How the Code is Structured

The program is structured using a **modular and procedural approach**, making the logic easier to understand, maintain, and expand. Below is a breakdown of the code structure:

1. Header Inclusions

#include <iostream>

#include <string>

- These libraries provide input/output functionality and access to the string class.

2. Namespace Declaration

using namespace std;

- Used to simplify syntax by removing the need to prefix standard library names with std::.

3. Function Definitions

a. encrypt(string text, int shift)

- This function handles **encryption logic**.
- It processes each character of the input string:
  - If it's a letter, it shifts it forward by the given number of positions.
  - Non-alphabetic characters are not altered.
- Returns the **encrypted string**.

b. decrypt(string text, int shift)

- This function handles **decryption logic**.
- It calls the encrypt() function with a **reversed shift** of 26 - shift, effectively undoing the original transformation.
- Returns the **original (decrypted) message**.

4. main() Function

- The **entry point** of the program.
- Handles all user interaction:
  - Takes user input for the message and shift key.
  - Validates the shift value (must be between 1 and 25).
  - Calls the encrypt() and decrypt() functions.
  - Displays the encrypted and decrypted results to the user.
- Provides a **simple user interface** via the console.

🧩 **Logical Flow**

1. **User Input → Encryption → Output Encrypted Text**
2. **Encrypted Text → Decryption → Output Original Text**

📌 **Summary of Structure**

Section	Purpose
Libraries	Enable core functionalities (I/O, strings).
Functions	Separate logic for encryption and decryption.
Main Function	Drives user interaction and function calls.
Modularity	Improves readability and makes future updates easier.

Code Listing

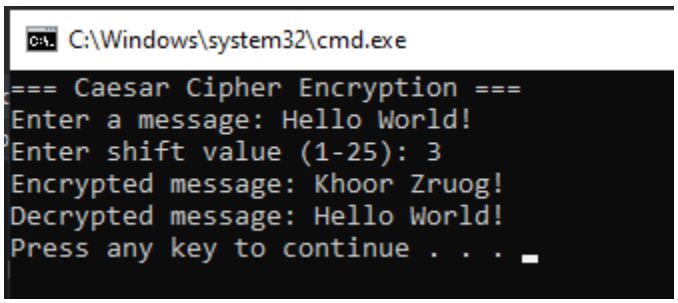
```
#include <iostream>  // For input/output operations
#include <string>    // To use the string class
using namespace std; // Avoids having to write std:: before standard functions
```

```
// Function to encrypt a string using Caesar Cipher
string encrypt(string text, int shift) {
  string result = "";  // Variable to store the encrypted result

  // Loop through each character in the input string
  for (char c : text) {
    if (isalpha(c)) {  // Check if the character is a letter
      // Determine if the character is uppercase or lowercase
      char base = isupper(c) ? 'A' : 'a';

      // Shift the character and wrap it around the alphabet
      result += char((c - base + shift) % 26 + base);
    } else {
      // If not an alphabet, keep the character unchanged
      result += c;
    }
  }
  return result;  // Return the final encrypted string
}

// Function to decrypt a Caesar Cipher encrypted string
string decrypt(string text, int shift) {
  // Decrypt by applying encryption with the reverse shift
  return encrypt(text, 26 - shift);
}
```

```
int main() {
    string message;  // Variable to hold user input message
    int shift;      // Variable to hold shift value

    cout << "=== Caesar Cipher Encryption ===" << endl;

    // Get the message from the user
    cout << "Enter a message: ";
    getline(cin, message);  // Read the full line including spaces

    // Get the shift value from the user
    cout << "Enter shift value (1-25): ";
    cin >> shift;

    // Validate the shift value
    if (shift < 1 || shift > 25) {
        cout << "Invalid shift. Must be between 1 and 25." << endl;
        return 1;  // Exit the program with an error code
    }

    // Encrypt the message using Caesar Cipher
    string encrypted = encrypt(message, shift);
    cout << "Encrypted message: " << encrypted << endl;

    // Decrypt the message back to the original
```

```
    string decrypted = decrypt(encrypted, shift);
    cout << "Decrypted message: " << decrypted << endl;

    return 0;  // Exit the program successfully
}
```

Output Screenshot



Explanation:

Here is the **Explanation of the Code Output** :

🖥️ Explanation of Code Output

When the program is executed, it interacts with the user via the console and performs the following operations:

✅ Step-by-Step Output Flow

1. **Prompt for Message**

Enter a message: Hello World!

- The program asks the user to enter the plaintext message. In this case, the user types Hello World!.

2. **Prompt for Shift Value**

Enter shift value (1-25): 3

- The user is asked to provide a shift value. Here, the user enters 3.

3. **Encrypted Output**

Encrypted message: Khoor Zruog!

- Each letter in the original message is shifted 3 positions forward in the alphabet:
  - H → K, e → h, l → o, etc.
  - Non-alphabetic characters like space and exclamation mark ! remain unchanged.

4. **Decrypted Output**

Decrypted message: Hello World!

- The program then reverses the shift to retrieve the original message using the decryption function.
- The result matches the original input, confirming correct functionality.

🎯 **Purpose of the Output**

- **Validates encryption logic**: The transformation of characters is accurately performed.
- **Confirms decryption**: Demonstrates that the program can revert to the original text.
- **Handles user input/output**: Ensures user-friendly interaction through prompts and clear output messages.

Conclusion

Summary:

The project successfully demonstrates the core concept of cryptography using Caesar's Cipher, a classical substitution encryption method. The implementation in C++ effectively performs both encryption and decryption based on a user-defined shift. Through modular design and clean logic, this project helped in reinforcing fundamental programming concepts like loops, conditionals, character manipulation, and function design.

Technical Achievements:

- Modular code structure with separate functions for encryption and decryption.
- Use of ASCII manipulation for character shifting while preserving case.
- Handles both uppercase and lowercase characters correctly.
- Ignores and retains non-alphabetic characters as-is.

Limitations:

- Only works with alphabetic characters (A–Z, a–z).
- Shift range must be between 1 and 25; no handling for 0 or negative shifts.
- Does not support Unicode or extended ASCII characters.

Future Enhancements:

- Extend support to full ASCII character set.
- Add input/output file handling for batch encryption.
- Implement a graphical user interface (GUI).
- Explore more advanced cryptographic methods like Vigenère Cipher or RSA encryption for enhanced security.

Overall, this project provided a foundational understanding of how substitution ciphers work and how such logic can be applied programmatically. It offers a practical insight into the principles of data security and serves as a stepping stone toward more complex encryption techniques.

References

- https://en.wikipedia.org/wiki/Caesar_cipher
- Programming tutorials on YouTube